The Current `Dynamics` Class

1 Purpose

The current `Dynamics` class is the collision-response kernel for the clean rebuild. It is intentionally separated from `Base`. The `Base` class owns the simulation loop, particle storage, contact detection, double-buffered position movement, and reporting. The `Dynamics` class owns only the math that converts detected contacts into momentum changes.
This separation is important because the dynamics code is the part most likely to be moved later into C++ or GLSL. The current design is therefore written with a simple kernel-like rule:

one worker owns one particle.

During collision processing, the worker for particle i may read selected state from another particle j, but it writes only to particle i.

2 Particle Data Contract

Each particle is currently stored as a Python dictionary. The dynamics class expects the following fields to exist before collision processing begins.

2.1 Position

Position is double-buffered:

$$\texttt{location} = \{\texttt{use}, \texttt{x1}, \texttt{y1}, \texttt{x2}, \texttt{y2}\}.$$

If

$$\texttt{use} = 1,$$

then the active position is

$$\mathbf{x} = (x_1, y_1).$$

If

$$\texttt{use} = 2,$$

then the active position is

$$\mathbf{x} = (x_2, y_2).$$

The helper `current_location()` reads this active position.

2.2 Particle Properties

The dynamics code reads:

$$r_i = \texttt{radius}, \qquad m_i = \texttt{mass}, \qquad \mathbf{v}_i = (v_{x,i}, v_{y,i}).$$

The current overlap-area formula assumes equal-radius disks:

$$r_i = r_j = r.$$

If unequal radii are passed to `circle_overlap_area()`, the code raises an error.

2.3 Contact List

`Base.detect_contacts()` fills each particle's contact storage before `Dynamics.process_collisions()` runs. For particle i, the relevant fields are:

$$\texttt{contact\_count}_i, \qquad \texttt{contacts}_i.$$

`contacts` is a fixed-length array of particle indices. Only the first `contact_count` entries are valid.

3 Memory Ownership Rule

The dynamics pass is particle-owned. When processing particle i, the code loops through i's own contacts:

$$j \in \texttt{contacts}_i.$$

The code may read particle j's current location and radius to calculate geometry. It does not write to particle j. This is different from a traditional pair solver that updates both particles in one pair operation.
The current update has the form:

$$\Delta\mathbf{p}_i = \sum_{j \in C_i} \Delta\mathbf{p}_{ij}.$$

The corresponding update for particle j is produced only when particle j's own worker processes its own contact list.

4 Contact Geometry

For a pair i, j, the active center positions are:

$$\mathbf{x}_i = (x_i, y_i), \qquad \mathbf{x}_j = (x_j, y_j).$$

The separation vector from particle i to particle j is:

$$\mathbf{d}_{ij} = \mathbf{x}_j - \mathbf{x}_i.$$

The center distance is:

$$d = \|\mathbf{d}_{ij}\|.$$

Contact exists when:

$$d < r_i + r_j.$$

If there is no overlap, `particle_contact()` returns `None`.

4.1 Contact Normal

When $d > 0$, the unit normal from particle i toward particle j is:

$$\mathbf{n}_{ij} = \frac{\mathbf{x}_j - \mathbf{x}_i}{d} = (n_x, n_y).$$

If the centers are effectively identical,

$$d \leq 10^{-12},$$

the code uses the deterministic fallback:

$$\mathbf{n}_{ij} = (1, 0).$$

This avoids division by zero.

5 Penetration Limit Used For Area

The raw overlap depth is:

$$\delta_{\text{raw}} = r_i + r_j - d.$$

The current code caps the depth used for area calculation:

$$\delta = \min\left(r_i + r_j - d, \min(r_i, r_j)\right).$$

Under the equal-radius assumption, this means:

$$\delta \leq r.$$

This matches the current development assumption that particles do not move past the case where the leading edge reaches the other particle center.
The distance sent to the overlap-area formula is:

$$d_A = r_i + r_j - \delta.$$

6 Equal-Radius Overlap Area

For equal-radius particles,

$$r_i = r_j = r,$$

the circular lens area is:

$$A(d_A) = 2r^2 \cos^{-1}\left(\frac{d_A}{2r}\right) - \frac{d_A}{2}\sqrt{4r^2 - d_A^2}.$$

The implementation clamps the distance into the valid range:

$$d_c = \min(\max(d_A, 0), 2r).$$

The actual computed area is:

$$A(d_c) = 2r^2 \cos^{-1}\left(\frac{d_c}{2r}\right) - \frac{d_c}{2}\sqrt{\max(0, 4r^2 - d_c^2)}.$$

The $\max(0, \cdot)$ term protects the square root from small floating-point roundoff errors.

7 Overlap Momentum

The current model follows the minimal legacy Mom-style idea:

$$\text{overlap area} \rightarrow \text{momentum amount}.$$

It does not treat overlap as a force integrated over the substep. There is no multiplication by Δt inside `overlap_momentum()`.
For a contact between particle i and particle j, the scalar momentum amount is:

$$J_{ij} = k_i A_{ij} w(d).$$

Here:

$$k_i = \texttt{momentum\_per\_area}$$

and:

$$A_{ij} = \text{overlap area}.$$

The coefficient k_i is owned by particle i. If particle i has a `momentum_per_area` field, that value is used. Otherwise the class default is used. The code also keeps the legacy fallback name `repulsion_force_per_area`.

8 Inverse-Square Weight

The distance weight is:

$$w(d) = \frac{1}{\max(d^2, s^2)}.$$

where

$$s = \texttt{inverse\_square\_softening}.$$

The softening value prevents singular behavior when two centers are very close:

$$s \geq 10^{-12}.$$

9 Momentum Direction

The contact normal $\mathbf{n}_{ij}$ points from particle i toward particle j. The collision response for particle i points away from particle j, so the update for this contact is:

$$\Delta \mathbf{p}_{ij} = -J_{ij}\mathbf{n}_{ij}.$$

Component-wise:

$$\Delta p_{x,ij} = -n_x J_{ij}, \qquad \Delta p_{y,ij} = -n_y J_{ij}.$$

10 Accumulating Contacts

For a particle with contact set C_i, the dynamics class accumulates:

$$P_{x,i} = \sum_{j \in C_i} -n_{x,ij} J_{ij},$$

$$P_{y,i} = \sum_{j \in C_i} -n_{y,ij} J_{ij}.$$

It also tracks the scalar sum:

$$P_{\text{sum},i} = \sum_{j \in C_i} J_{ij}.$$

These values are written back onto particle i as:

$$\begin{aligned} \texttt{overlap\_momentum\_x} &= P_{x,i}, \\ \texttt{overlap\_momentum\_y} &= P_{y,i}, \\ \texttt{overlap\_momentum} &= P_{\text{sum},i}. \end{aligned}$$

The same vector is currently also stored as internal momentum:

$$\begin{aligned} \texttt{internal\_momentum\_x} &= P_{x,i}, \\ \texttt{internal\_momentum\_y} &= P_{y,i}, \\ \texttt{internal\_momentum} &= \sqrt{P_{x,i}^2 + P_{y,i}^2}. \end{aligned}$$

11 Velocity Update

The method `apply_overlap_momentum()` converts the stored momentum vector into a velocity change:

$$\Delta \mathbf{v}_i = \frac{\Delta \mathbf{p}_i}{m_i}.$$

Component-wise:

$$v_{x,i}^{\text{new}} = v_{x,i} + \frac{\texttt{overlap\_momentum\_x}}{m_i},$$

$$v_{y,i}^{\text{new}} = v_{y,i} + \frac{\texttt{overlap\_momentum\_y}}{m_i}.$$

The position update for the current substep happens in `Base` before collision processing. Therefore this velocity change affects the next movement step.

12 Current Step Order

At the current rebuild stage, the relevant order is:

1. `Base` predicts the next position using current velocity.
2. `Base.move()` flips the active location buffer.
3. `Base.detect_contacts()` fills each particle's contact list.
4. `Dynamics.process_collisions()` computes overlap momentum.
5. `Dynamics.apply_overlap_momentum()` updates velocity.
6. `Base.record_step_report()` stores per-particle values.

13 Estimating Steps Spent In Collision

For debugging and stability checks, it is useful to estimate how many simulation steps a pair of particles will spend in overlap. This estimate helps identify cases where the relative motion is so fast that contact may last only one or two sampled steps.
Let the frame rate be F frames per second and let the simulation use N_s substeps per frame. The substep duration is:

$$\Delta t_s = \frac{1}{FN_s}.$$

For two particles with positions $\mathbf{x}_i, \mathbf{x}_j$, velocities $\mathbf{v}_i, \mathbf{v}_j$, and radii r_i, r_j, define the relative position and velocity:

$$\mathbf{x}_{rel} = \mathbf{x}_j - \mathbf{x}_i, \qquad \mathbf{v}_{rel} = \mathbf{v}_j - \mathbf{v}_i.$$

The pair is geometrically in contact while:

$$\|\mathbf{x}_{rel} + \mathbf{v}_{rel} t\| \leq r_i + r_j.$$

Expanding this condition gives a quadratic:

$$at^2 + bt + c \leq 0,$$

where:

$$\begin{aligned} a &= \mathbf{v}_{rel} \cdot \mathbf{v}_{rel}, \\ b &= 2(\mathbf{x}_{rel} \cdot \mathbf{v}_{rel}), \\ c &= \mathbf{x}_{rel} \cdot \mathbf{x}_{rel} - (r_i + r_j)^2. \end{aligned}$$

If the discriminant

$$D = b^2 - 4ac$$

is negative, the constant-velocity paths do not overlap. If $D \geq 0$, the entry and exit times are:

$$t_{entry} = \frac{-b - \sqrt{D}}{2a}, \qquad t_{exit} = \frac{-b + \sqrt{D}}{2a}.$$

The continuous collision duration is:

$$T_c = t_{exit} - t_{entry}.$$

The estimated number of simulation substeps spent in collision is:

$$n_c = T_c F N_s = \frac{T_c}{\Delta t_s}.$$

13.1 Speed-Limit Estimate

If the only known velocity is a system speed limit v_{max}, the largest possible relative speed for two particles moving directly toward each other is:

$$v_{rel,max} = 2v_{max}.$$

The shortest head-on geometric contact duration is therefore:

$$T_{c,min} = \frac{2(r_i + r_j)}{v_{rel,max}} = \frac{r_i + r_j}{v_{max}}.$$

The corresponding conservative lower bound on substeps in collision is:

$$n_{c,min} = \frac{r_i + r_j}{v_{max}} F N_s.$$

For equal-radius particles $r_i = r_j = r$:

$$n_{c,min} = \frac{2r}{v_{max}} F N_s.$$

This estimate ignores collision response. It only answers how many sampled substeps the particles would overlap if they continued at constant velocity.

13.2 Sampling Risk

A useful diagnostic is the relative distance traveled in one substep:

$$\Delta x_{rel,s} = \|\mathbf{v}_{rel}\| \Delta t_s.$$

If:

$$\Delta x_{rel,s} > r_i + r_j,$$

then the particles move more than the contact radius in one substep. If:

$$\Delta x_{rel,s} > 2(r_i + r_j),$$

then they can cross the entire geometric collision window in one substep. These cases are risky for overlap-only contact detection because a collision may be sampled for too few steps, or missed entirely depending on the sampled positions.

13.3 Response-Aware Estimate

The actual number of steps in contact can differ from the geometric estimate because `momentum_per_area` and `inverse_square_softening` change particle velocities during overlap. For each substep, the current model computes:

$$J_{ij} = k_i A_{ij} w(d),$$

with:

$$k_i = \texttt{momentum\_per\_area}, \qquad w(d) = \frac{1}{\max(d^2, s^2)}, \qquad s = \texttt{inverse\_square\_softening}.$$

The response velocity change is:

$$\Delta \mathbf{v}_i = \frac{-J_{ij}\mathbf{n}_{ij}}{m_i}.$$

Because this velocity change alters later positions, a response-aware collision step count is best estimated by running the same substep loop as the simulation:

1. compute the current center distance,
2. detect whether $d < r_i + r_j$,
3. compute overlap area A_{ij},
4. compute $J_{ij} = k_i A_{ij} w(d)$,
5. update velocity from momentum,
6. move by the substep duration Δt_s,
7. count each substep where overlap is present.

The geometric estimate and the response-aware estimate should both be reported. The geometric estimate shows whether the time resolution is adequate; the response-aware estimate shows how the current momentum parameters change the actual contact duration.

13.4 Issues

The are a number of issues that arise while designing a simulation.
(1) There is an python menu driven application velocity_menu.py that takes parameters of the maximum speed and radius of particle which produces the number of frames the particle will be in collision. If there are errors (tunneling etc.) it will report these also.
(2) Reducing the particle radius requires increasing momentum per area and increasing the radius requires lowering it.

14 What This Class Does Not Do

The current `Dynamics` class does not include bonding, field dynamics, long-term internal momentum release, classical restitution coefficient e, or pair-store bookkeeping. It is only the minimal overlap-momentum collision kernel.
It also does not update both particles inside a single pair operation. That is intentional for the current particle-owned memory model.

15 Future Porting Notes

The most direct C++ or GLSL version would give each particle one worker. That worker would:

1. read particle i's contact count and contact array,
2. loop through the valid contact slots,
3. read particle j's current position and radius,
4. compute A_{ij}, J_{ij}, and $-J_{ij}\mathbf{n}_{ij}$,
5. write only particle i's momentum outputs and updated velocity.

The current Python class is shaped to keep that translation straightforward.